8c. Type Stability with Scalars and Vectors

Martin Alfaro
PhD in Economics

INTRODUCTION

The **previous section** has defined type stability, along with approaches to checking whether the property holds. The formal definition of a type-stable function is that the function's output type can be inferred from its argument types. In practice, however, we often rely on a more stringent definition, which requires that the compiler can infer a single concrete type for each expression within the function body. This property guarantees that every operation is specialized, resulting in optimal performance. [1]

In this section, we start the analysis of type stability for specific objects. We cover in particular the case of scalars and vectors, providing practical guidance for achieving type stability with them.

TYPES OF SCALARS AND VECTORS

The notion of type stability applied to scalars is straightforward. It demands operations to be performed on variables with the same concrete type (e.g., `Float64`, `Int64`, `Bool`). Likewise, type stability for vectors requires that their *elements* have a concrete type.

The following table identifies types for scalars and vectors satisfying this property.

Objects Whose Elements Have Concrete Types

Scalars	Vectors
`Int`	`Vector{Int}`
`Int64`	`Vector{Int64}`
`Float64`	`Vector{Float64}`
`Bool`	`BitVector`

Note: `Int` defaults to `Int64` or `Int32`, depending on the CPU architecture.

Next, we'll delve into type stability in scalars and vectors, considering each case separately.

TYPE STABILITY WITH SCALARS

To make the definition of type stability for scalars operational, let's revisit some concepts about types. Recall that only concrete types such as `Int64` or `Float64` can be **instantiated**. Moreover, every value that appears during execution is ultimately an instance of one of these concrete types. Instead, abstract types like `Any` or `Number` can't be instantiated, meaning they never hold values directly. Their role is organizational: they describe sets of possible concrete types and provide the structure that shapes the type hierarchy.

This distinction explains why a type annotation such as `x::Number` shouldn't be read as `x` having type `Number`. Rather, it constrains `x` to values whose concrete types are subtypes of `Number`. At runtime `x` must always have a concrete type. For instance, after evaluating `x::Number = 2`, the variable `x` contains the value 2, whose concrete type is `Int64`.

With this in mind, we can discuss how type instability arises. A common source of instability is mixing values of different types, such as combinations of `Int64` with `Float64`. However, mixing types doesn't automatically imply type instability. This leads us to define the concepts of type promotion and conversion.

TYPE PROMOTION AND CONVERSION

Julia employs various mechanisms to handle cases that combine `Int64` and `Float64`. The first one is part of a concept known as **type promotion**, which converts dissimilar types to a common one whenever possible. The second one emerges when variables are type-annotated, in which case Julia engages in **type conversion**. By transforming values to the respective type declared, this feature also prevents the mix of types. Both mechanisms are illustrated below.

TYPE PROMOTION

```
foo(x,y)     = x * y

x1           = 2
y1           = 0.5

output         = foo(x1,y1)        # type stable: mixing `Int64` and `Float64` results in
`Float64`
```

```
julia> output
1.0
```

TYPE CONVERSION

```
foo(x,y)     = x * y

x2::Float64 = 2                # this is converted to `2.0`
y2          = 0.5

output       = foo(x2,y2)       # type stable: `x` and `y` are `Float64`, so output type is
predictable
```

```
julia> output
1.0
```

In the first tab, the output type depends on the types of the function arguments. However, in all cases the output type can be predicted, since mixing `Int64` and `Float64` results in `Float64` due to automatic type promotion.

As for the second tab, Julia transforms the value of `x2` to make it consistent with the type-annotation declared. Consequently, `x * y` is computed as the product of two values with type `Float64`.

TYPE INSTABILITY WITH SCALARS

While type promotion and conversion can handle several situations, they certainly don't cover all cases. One such scenario is when a scalar value depends on a conditional expression and each branch returns values of different types. In this situation, since the compiler reasons about types rather than values, it can't determine which branch is relevant for the function call. As a result, it'll generate code that accommodates both possibilities, as it happens in the following example.

INT64

```
function foo(x,y)
    a = (x > y) ?  x  :  y

    [a * i for i in 1:100_000]
end

foo(1, 2)            # type stable   -> `a * i` is always `Int64`
```

```
julia> @btime foo(1,2)
  17.949 µs (3 allocations: 781.320 KiB)
```

MIXING INT64 AND FLOAT64

```julia
function foo(x,y)
    a = (x > y) ?  x  :  y

    [a * i for i in 1:100_000]
end

foo(1, 2.5)         # type UNSTABLE -> `a * i` is either `Int64` or `Float64`
```

```
julia> @btime foo(1,2.5)
  38.658 μs (3 allocations: 781.320 KiB)
```

In the example, type instability is inevitable if `x` and `y` have different types. The reason is that type promotion only ensures that `a * i` will be converted to `Float64` if `a` is `Float64`. However, the compiler still needs to consider the possibility that `a` could be `Int64`, in which case `a * i` would be `Int64`.

Since both outcomes are possible from the compiler's perspective, the generated method instance has to support both. Only at runtime can Julia observe the actual types, narrow down the possibilities, and select the appropriate computation implementation.

TYPE STABILITY WITH VECTORS

Vectors in Julia are collections whose elements share the same element type. Given this definition, reasoning about type stability requires distinguishing between the type of the vector itself and the type of its elements.

Recall that, by construction, **every vector type is concrete**. For instance, `Vector{Any}` is a concrete type, even though its elements may belong to any subtype of `Any`. Consequently, this fact can't serve as a criterion for type stability, since otherwise all functions operating on vectors would automatically be type stable. In fact, a function operating on a vector like `Vector{Any}` would lead to type instability if not addressed properly.

Instead, what matters is that operations on vectors ultimately manipulate individual elements. Thus, **type stability is contingent on whether their element type is concrete**. [2]

TYPE INSTABILITY

Below, we compare type stability for vectors, depending on whether their element types are abstract or concrete. All examples rely on the function `sum`.

We begin with vectors whose element types are concrete.

VECTOR{INT64}

```
z1::Vector{Int64}   = [1, 2, 3]

sum(z1)             # type stable
```

VECTOR{FLOAT64}

```
z2::Vector{Float64} = [1, 2, 3]

sum(z2)             # type stable
```

BITVECTOR

```
z3::BitVector       = [true, false, true]

sum(z3)             # type stable
```

In contrast, the next examples use vectors whose elements have abstract types. They result in type instability.

VECTOR{NUMBER}

```
z4::Vector{Number} = [1, 2, 3]

sum(z4)                 # type UNSTABLE -> `sum` must consider all possible subtypes of
`Number`
```

VECTOR{ANY}

```
z5::Vector{Any}    = [1, 2, 3]

sum(z5)             # type UNSTABLE -> `sum` must consider all possible subtypes of `Any`
```

TYPE PROMOTION AND CONVERSION

By definition, vectors require all their elements to share the same type. This means that, when creating a vector, Julia will attempt to identify a type that can accommodate every element. This common type can be either abstract or concrete, depending on what the elements allow. For example, if you mix elements with disparate types, such as `String` and `Int64`, Julia will infer the vector's type as `Vector{Any}`. This will pose a problem for type stability.

In some cases, nonetheless, the elements can be converted to a shared concrete type. For example, this is the case when mixing `Float64` and `Int64`, in which case Julia will perform a conversion to `Float64` automatically. The following example illustrates this mechanism in a simple assignment.

AUTOMATIC CONVERSION

```julia
x = [1, 2, 2.5]      # automatic conversion to `Vector{Float64}`
```

```
julia> x
3-element Vector{Float64}:
 1.0
 2.0
 2.5
```

AUTOMATIC CONVERSION

```julia
y = [1, 2.0, 3.0]    # automatic conversion to `Vector{Float64}`
```

```
julia> y
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

Likewise, when assignments are declared with type-annotations and values differ in type, Julia will attempt to convert all values to the declared type.

ASSIGNMENT WITH CONVERSION

```julia
v1                = [1, 2.0, 3.0]     # automatic conversion to `Vector{Float64}`

w1::Vector{Int64}  = v1                # conversion to `Vector{Int64}`
```

```
julia> w1
3-element Vector{Int64}:
 1
 2
 3
```

ASSIGNMENT WITH NO CONVERSION

```julia
v2                = [1, 2, 2.5]       # automatic conversion to `Vector{Float64}`

w2::Vector{Number} = v2                # `w2` is still `Vector{Number}`
```

```
julia> w2
3-element Vector{Number}:
 1.0
 2.0
 2.5
```

FOOTNOTES

[1.] Nevertheless, simply demanding that the output's type can be inferred from the input types already offers benefits. In particular, it ensures that type instability won't be propagated when the function is called in other operations.

[2.] Note that this condition isn't sufficient to guarantee type stability. Ultimately, whether a function is type stable still depends on how it's implemented. In practice, though, functions in well-designed packages are typically type stable if they receive vectors whose elements have a concrete type.